11. How is load balancing maintained in Kafka?

Load balancing in Kafka is handled by the producers. The message load is spread out between the various partitions while maintaining the order of the message. By default, the producer selects the next partition to take up message data using a round-robin approach. If a different approach other than round-robin is to be used, users can also specify exact partitions for a message.

14. Explain the retention period in an Apache Kafka cluster.

Messages sent to Kafka clusters get appended to one of the multiple partition logs. These messages remain in multiple partition logs even after being consumed, for a configurable period of time, or until a configurable size is reached. This configurable amount of time for which the message remains in the log is known as the retention period. The message will be available for the amount of time specified by the retention period. Kafka allows users to configure the retention period for messages on a per-topic basis. The default retention period for a message is seven days.

22. Explain fault tolerance in Apache Kafka.

In Kafka, the partition data is copied to other brokers, which are known as replicas. If there is a point of failure in the partition data in one node, then there are other nodes that will provide a backup and ensure that the data is still available. This is how Kafka allows fault tolerance.

23. What is the importance of replication in Kafka?

In Kafka, replication provides fault tolerance by ensuring that published messages are not permanently lost. Even if a node fails and they are lost on one node due to program error, machine error, or even due to software upgrades, then there is a replica present on another node that can be recovered.

29. Mention some disadvantages of Apache Kafka.

- Tweaking of messages in Kafka causes performance issues in Kafka. Kafka works well in cases where the message does not need to be changed.
- Kafka does not support wildcard topic selection. The exact topic name has to be matched.
- In the case of large messages, brokers and consumers reduce the performance of Kafka by compressing and decompressing the messages. This affects the throughput and performance of Kafka.
- Kafka does not support certain message paradigms like a point-to-point queue and client request/reply.

32. Mention some of the system tools available in Apache Kafka.

The three main system tools available in Apache Kafka are:

Kafka Migration Tool − This tool is used to migrate the data in a Kafka cluster from one version to another.

Kafka MirrorMaker − This tool copies data from one Kafka cluster to another.

Consumer Offset Checker − This tool displays the Consumer Group, Topic, Partitions, Off-set, logSize, and Owner for specific Topics and Consumer Group sets.

37. How can Kafka be tuned for optimal performance?

Tuning for optimal performance involves consideration of two key measures: latency measures, which denote the amount of time taken to process one event, and throughput measures, which refer to how many events can be processed in a specific time. Most systems are optimized for either latency or throughput, while Kafka can balance both. Tuning Kafka for optimal performance involves the following steps:

- Tuning Kafka producers: Data that the producers have to send to brokers is stored in a batch. When the batch is ready, the producer sends it to the broker. For latency and throughput, to tune the producers, two parameters must be taken care of: batch size and linger time. The batch size has to be selected very carefully. If the

producer is sending messages all the time, a larger batch size is preferable to maximize throughput. However, if the batch size is chosen to be very large, then it may never get full or take a long time to fill up and, in turn, affect the latency. Batch size will have to be determined, taking into account the nature of the volume of messages sent from the producer. The linger time is added to create a delay to wait for more records to get filled up in the batch so that larger records are sent. A longer linger time will allow more messages to be sent in one batch, but this could compromise latency. On the other hand, a shorter linger time will result in fewer messages getting sent faster - reduced latency but reduced throughput as well.

- Tuning Kafka broker: Each partition in a topic is associated with a leader, which will further have 0 or more followers. It is important that the leaders are balanced properly and ensure that some nodes are not overworked compared to others.
- Tuning Kafka Consumers: It is recommended that the number of partitions for a topic is equal to the number of consumers so that the consumers can keep up with the producers. In the same consumer group, the partitions are split up among the consumers.

38. What are the differences between Redis and Kafka?

Redis is the short form for remote dictionary servers. It is a key-value store and can be used as a repository to read and write requests. Redis is a no-SQL database.

Redis	Apache Kafka
Redis supports push-based message delivery. This means that messages published to Redis will be automatically delivered to the consumers.	Apache Kafka supports the pull-based delivery of messages. The messages published to the Kafka broker are not delivered automatically to the consumers; rather, the consumers have to pull the messages when they are ready for them.
Redis does not support parallel processing.	Due to the partitioning system in Apache Kafka, one or more consumers of

	a specific consumer group can parallelly consume partitions of the topic at the same time.
Redis does not support message replicas. Once the messages are delivered to the consumers, they are deleted.	Apache Kafka supports creating message replicas in its log.
Redis is an in-memory store, which makes it faster than Kafka.	Apache Kafka uses disk space for storage, which makes it slower than Redis.
Since Redis is an in-memory store, it cannot handle large volumes of data.	Since Kafka uses disk space as its primary storage, it is capable of handling large volumes of data.

39. Is it possible to add partitions to an existing topic in Apache Kafka?

Apache Kafka provides the alter command to change a topic behavior and modify the configurations associated with it. The alter command can be used to add more partitions.

The command to increase the partitions to four is as follows:

./bin/kafka-topics.sh --alter --zookeeper localhost:2181 --topic my-topic --partitions 4

40. What is the optimal number of partitions for a topic?

The optimal number of partitions a topic should be divided into must be equal to the number of consumers.

41. How does one view a Kafka message?

The Kafka-console-consumer.sh command can be used to view the messages. The following command can be used to view the messages from a topic:

bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test --from-beginning

42. How can all brokers available in a cluster be listed?

Two ways to get the list of available brokers in an Apache Kafka cluster are as follows:

- Using zookeeper-shell.sh

zookeeper-shell.sh :2181 ls /brokers/ids

Which will give an output like:

WATCHER:: WatchedEvent state:SyncConnected type:None path:null [0, 1, 2, 3]

This indicates that there are four alive brokers - 0,1,2 and 3

- Using zkCli.sh

  First, you have to log in to the ZooKeeper client

  zkCli.sh -server :2181

Then use the below command to list all the available brokers

ls /brokers/ids

Both the methods used above make use of the ZooKeeper to find out the list of available brokers.

43. What is the Kafka MirrorMaker?

The Kafka MirrorMaker is a stand-alone tool that allows data to be copied from one Apache Kafka cluster to another. The Kafka MirrorMaker will read data from topics in the original cluster and write the topics to a destination cluster with the same topic name. The source and destination clusters are independent entities and can have different numbers of partitions and varying offset values.

44. What is the role of the Kafka Migration Tool?

The Kafka Migration tool is used to efficiently move from one environment to another. It can be used to move existing Kafka data from an older version of Kafka to a newer version.

46. How can you list the topics being used in Apache Kafka?

Once you start the ZooKeeper, you can list all the topics using

bin/kafka-topics.sh --list --zookeeper localhost:2181

47. What are name restrictions for Kafka topics?

According to Apache Kafka, there are some legal rules to be followed to name topics, which are as follows:

- The maximum length is 255 characters (symbols and letters). The length has been reduced from 255 to 249 in Kafka 0.10
- . (dot), _ (underscore), - (hyphen) can be used. However, topics with a dot (.) and underscore (_) could cause some confusion with internal data structures, and hence, it is advisable to use either but not both.

48. What is the Confluent Replicator?

The Confluent Replicator allows easy and reliable replication of topics from a source cluster to a destination cluster. It continuously copies messages from the source to the destination and even assigns the same names to the topics in the destination cluster.

49. How can load balancing be ensured in Apache Kafka when one Kafka fails?

When a Kafka server fails, if it was the leader for any partition, one of its followers will take on the role of being the new leader to ensure load balancing. For this to happen, the topic replication factor has to be more than one, i.e., the leader should have at least one follower to take on the new role of leader.

50. Where is the meta-information about topics stored in the Kafka cluster?

Currently, in Apache Kafka, meta-information about topics is stored in the ZooKeeper. Information regarding the location of the partitions and the configuration details related to a topic are stored in the ZooKeeper in a separate Kafka cluster.

51. How can large messages be sent in Apache Kafka?

By default, the largest size of a message that can be sent in Apache Kafka is 1MB. In order to send larger messages using Kafka, a few properties have to be adjusted. Here are the configuration details that have to be updated

- At the Consumer end – fetch.message.max.bytes
- At the Broker, end to create replica– replica.fetch.max.bytes
- At the Broker, the end to create a message – message.max.bytes
- At the Broker end for every topic – max.message.bytes

52. Explain the scalability of Apache Kafka.

In software terms, the scalability of an application is its ability to maintain its performance when it is exposed to changes in application and processing demands. In Apache Kafka, the messages corresponding to a particular topic are divided into partitions. This allows the topic size to be scaled beyond the size that will fit on a single server. Allowing a topic to be divided into partitions ensures that Kafka can guarantee load balancing over multiple consumer processes. In addition, the concept of the consumer group in Kafka also contributes to making it more scalable. In a consumer group, a particular partition is consumed by only one consumer in the group. This aids in the parallelism of consuming multiple messages on a topic.

53. What is the command to start ZooKeeper?

bin/zookeeper-server-start.sh

54. Explain how topics can be added and removed.

To create a topic:

```
kafka/bin/kafka-topics.sh --create \
--zookeeper localhost:2181 \
--replication-factor [replication factor] \
--partitions [number_of_partitions] \
--topic [unique-topic-name]
```

To Delete a topic:

- Go to ${kafka_home}/config/server.properties, and add the below line:

Delete.topic.enable = true

- Start the Kafka server once again with the new configuration:

${kafka_home}/bin/kafka-server-start.sh
~/kafka/config/server.properties

- Delete the topic:

${kafka_home}/bin/kafka-topics.sh --delete --zookeeper localhost:2181 --topic topic-name

55. Explain how topic configurations can be modified in Apache Kafka.

To add a config:

bin/kafka-configs.sh --zookeeper localhost:2181 --topics --topic_name --alter --add-config x=y

To remove a config:

bin/kafka-configs.sh --zookeeper localhost:2181 --topics --topic_name --alter --delete-config x

Where x is the particular configuration key that has to be changed.

56. Mention some differences between Kafka and JMS.

Kafka	JMS (Java Messaging Service)
The delivery system is based on a pull mechanism. Consumers pull the messages when they are ready to consume them.	The message delivery system is based on a push model. Messages are automatically delivered to consumers.
Messages are retained for a specific period of time, even after the consumer reads the messages.	Once the JMS queue receives an acknowledgment that the consumer has received the message, it gets permanently deleted.

Kafka guarantees that the partitions are delivered in the order that they were present in the message.	JMS is a queue that works on the FIFO system; it does not support any other form of order.
Kafka is more suitable for handling a large volume of data.	JMS is more suitable in highly complex systems with multi-node clusters.

57. What is meant by Kafka Connect?

Kafka Connect is a tool provided by Apache Kafka to allow scalable and reliable streaming data to move between Kafka and other systems. It makes it easier to define connectors that are responsible for moving large collections of data in and out of Kafka. Kafka Connect is able to process entire databases as input. It can also collect metrics from application servers into Kafka topics so that this data can be available for Kafka stream processing.

58. Explain message compression in Apache Kafka.

In Apache Kafka, producer applications write data to the brokers in JSON format. The data in the JSON format is stored in string form, which can result in several duplicated records getting stored in the Kafka topic. This leads to an increased occupation of disk space. Hence, to reduce this disk space, compression of messages or lingering the data is performed before sending the messages to Kafka. Message compression is done on the producer side, and hence there is no need to make any changes to the configuration of the consumer or the broker.

59. What is the need for message compression in Apache Kafka?

Message compression in Kafka does not require any changes in the configuration of the broker or the consumer. It is beneficial for the following reasons:

- Due to reduced size, it reduces the latency in which messages are sent to Kafka.
- Reduced bandwidth allows the producers to send more net messages to the broker.
- When the data is stored in Kafka via cloud platforms, it can reduce the cost in cases where the cloud services are paid.
- Message compression leads to reduced disk load, which will lead to faster read and write requests.

60. What are some disadvantages of message compression in Apache Kafka?

- Producers end up using some CPU cycles for compression.
- Consumers use some CPU cycles for decompression.
- Compression and decompression result in greater CPU demand.

61. Explain producer batch in Apache Kafka.

Producers write messages to Kafka, one at a time. Kafka waits for the messages that are being sent to Kafka, creates a batch and puts the messages into the batch, and waits until this batch becomes full. Only then is the batch sent to Kafka. The batch here is known as the producer batch. The default size of a producer batch is 16KB, but it can be modified. The larger the batch size, the more is the compression and throughput of the producer requests.

62. Highlight some differences between Kafka Streams and Spark Streaming.

Kafka streams	Spark Streaming
Able to handle only real-time streams	Can handle real-time streams as well as batch processes.
The use of partitions and their replicas allows Kafka to be fault-tolerant.	Spark allows recovery of partitions using Cache and RDD (resilient distributed dataset)
Kafka does not provide any interactive modes. The broker simply consumes the data from the producer and waits for the client to read it.	Has interactive modes
Messages remain persistent in the Kafka log.	A dataframe or some other data structure has to be used to keep the data persistent.

63. Explain how Apache Kafka provides security.

There are three components to the security provided by Kafka:

- Encryption: All the message transfer processes between the Kafka broker and its various clients are secured through encryption. This ensures that other clients cannot intercept the data. All the messages are shared between the components in an encrypted format.
- Authentication: applications that are making use of the Kafka broker have to be authenticated before they can be connected to Kafka. Only authorized applications will be allowed to consume or publish messages. Authorized applications will have unique ids and passwords to identify themselves.
- Authorization: this is done after authentication. Once a client is authenticated, it is allowed to consume or publish messages. The

authorization ensures that applications can be restricted from write access to prevent data pollution.

64. Can a consumer read more than one partition from a topic?

Yes, if the number of partitions is greater than the number of consumers in a consumer group, then a consumer will have to read more than one partition from a topic.

65. Can Apache Kafka be considered to be a distributed streaming platform? Elaborate.

Yes, Apache Kafka is considered to be a distributed streaming platform. A streaming platform can be called such if it has the following three capabilities:

- Be able to publish and subscribe to streams of data.
- Provide services similar to that of a message queue or scalable enterprise messaging system.
- Store streams of records in a durable and fault-tolerant manner.

Since Kafka meets all three of these requirements, it can be considered to be a streaming platform.

Furthermore, since a Kafka cluster consists of multiple servers that function as brokers, it is said to be distributed. Kafka topics are divided into multiple partitions to ensure load balancing. Brokers process these partitions parallelly and allow multiple producers and consumers to publish and retrieve messages in parallel.

Distributed streaming platforms handle large amounts of data in real-time by pushing them to multiple servers for real-time processing.

66. Mention some use cases where Apache Kafka is not suitable.

- Kafka is built to handle high volumes of data. If the requirement is to process only a small number of messages per day, a traditional messaging system would be more suitable.
- Kafka has a streaming API, but it is not suitable for performing data transformation operations. Kafka is to be avoided for ETL (extract, transform, load) jobs.
- In cases where a simple task queue is needed, there are better alternatives like the RabbitMQ.
- Kafka is not good if long-term storage is required. It supports saving data only for a specified retention period and not longer than that.

67. Define consumer lag in Apache Kafka.

Consumer lag refers to the lag between the Kafka producers and consumers. Consumer groups will have a lag if the data production rate far exceeds the rate at which the data is getting consumed. Consumer lag is the difference between the latest offset and the consumer offset.

68. What guarantees does Kafka provide?

- Consumers can see the messages in the same sequence in which the producers published them. The messaging order is preserved.
- The replication factor determines the number of replicas. If the replication factor is n, then there is a fault tolerance for up to n-1 servers in the Kafka cluster.
- Kafka can guarantee “at least one” delivery semantics per partition. This means that for multiple attempts at delivering a partition, Kafka guarantees that it will be delivered to a consumer at least once.

What is the primary purpose of log compaction in Kafka? How does log compaction impact the performance of Kafka consumers?

The main goal of log compaction in Kafka is to retain the most recent value for each unique key in a topic's log, ensuring that the latest state of the data is preserved and reducing storage usage. This allows consumers to access the current value more efficiently without having to process older duplicates.

Unlike Kafka's traditional retention policy, which deletes messages after a certain time period, log compaction deletes older records only for each key, retaining the latest value for that key. This feature helps ensure consumers always have access to the current state while maintaining a compact log for better storage efficiency and faster lookups.

70. What do you understand about quotas in Kafka?

As of Kafka 0.9, a Kafka cluster can enforce quotas on producers and fetch any client request. Quotas are byte-rate thresholds that are defined per client-id. A client-id is used to identify an application making a client request logically. A single client-id can refer to multiple producers and consumer instances. The quota will apply to all of them as a single entity. Quotas ensure that a single application does not monopolize the broker resources and cause network saturation by consuming very high volumes of data.

71. What is meant by cluster id in Kafka?

Kafka clusters are assigned unique and immutable identifiers. The identifier for a particular cluster is known as the cluster id. A cluster id can have a maximum number of 22 characters and has to follow the regular expression [a-zA-Z0-9_\-]+. It is generated when a broker with version 0.10.1 or later is successfully started for the first time. The broker attempts to get the cluster id from the znode during startup. If the znode does not exist, the broker generates a new cluster id and creates a znode with this cluster id.

72. Can Apache Kafka be integrated with Apache Storm? If yes, explain how.

Yes, Apache Kafka and Apache Storm naturally complement each other. Apache Storm is a distributed real-time processing system that allows the processing of very large amounts of data. Storm runs continuously consuming data from configured sources and passes it along the data pipeline to configured destinations.

In Storm, for streaming data processing, the following components work together:

- Spout: source of the stream. It is a continuous stream of log data

- Bolt: the bolt consumes input streams, processes them, and possibly emits new streams.

Here are some of the classes that can be used to integrate Apache Storm and Apache Kafka:

BrokerHosts:

BrokerHosts is an interface. ZkHosts and StaticHosts are two of their implementations. ZkHosts tracks the Kafka brokers dynamically and is used to maintain their details in the ZooKeeper. StaticHosts is used to manually set the Kafka brokers and their details.

SpoutConfig:

This class is an extension of the KafkaConfig class that supports additional ZooKeeper information. Its signature is as follows:

public SpoutConfig(BrokerHosts hosts, string topic, string zkRoot, string id)

Where:

- hosts: any implementation of BrokerHosts interface
- Topic: Name of the topic
- zkRoot: root path of the ZooKeeper
- Id: the spout stores the state of the offset that it has consumed in ZooKeeper. 'Id' here should uniquely identify the spout.

KafkaSpout API

KafkaSpout is our spout implementation, which will be integrated with Storm. It fetches the messages from the Kafka topic and emits them into the Storm ecosystem as tuples. KafkaSpout gets its configuration details from SpoutConfig.

IRichBolt

Bolt creation is done using the IRichBolt interface. Bolts take tuples as input, process the tuples, and produce new tuples as the output.

IRichBolt interface has the methods listed below:

- Prepare - this provides the bolt with an environment to execute. An executor can call this method to initialize the spout.

- Execute: this is used to process a single tuple of input.
- Cleanup - this is called when a bolt is ready to be shut down.
- declareOutputFields - this is used to specify the output schema of the tuple.

73. Why is the Kafka broker said to be “dumb”?

The Kafka broker does not keep a tab of which the consumers have read messages. It simply keeps all of the messages in its queue for a fixed time, known as the retention time, after which the messages are deleted. It is the responsibility of the consumer to pull the messages from the queue. Hence, Kafka is said to have a “smart-client, dumb-broker” architecture.

74. When does Kafka throw a BufferExhaustedException?

BufferExhaustedException is thrown when the producer cannot allocate memory to a record due to the buffer being too full. The exception is thrown if the producer is in non-blocking mode and the rate of data production exceeds the rate at which data is sent from the buffer for long enough for the allocated buffer to be exhausted.

75. What are the responsibilities of a Controller Broker in Kafka?

The main role of the Controller is to manage and coordinate the Kafka cluster, along with the Apache ZooKeeper. Any broker in the cluster can take on the role of the controller. However, once the application starts running, there can be only one controller broker in the cluster. When the broker starts, it will try to create a Controller node in ZooKeeper. The first broker that creates this controller node becomes the controller.

The controller is responsible for:

- creating and deleting topics
- Adding partitions and assigning leaders to the partitions
- Managing the brokers in a cluster - adding new brokers, active broker shutdown, and broker failures
- Leader Election
- Reallocation of partitions.

76. What causes OutOfMemoryException?

OutOfMemoryException can occur if the consumers are sending large messages or if there is a spike in the number of messages wherein the consumer is sending messages at a rate faster than the rate of downstream processing. This causes the message queue to fill up, taking up memory.

77. How can Kafka retention time be changed at runtime?

The retention time can be configured in Kafka for a topic. The default retention time for a topic is seven days. The retention time can be configured while a new topic is set up. Log.retention.hours is the property of a broker, which is used to set the retention time when a topic is created. However, when configurations have to be changed for a currently running topic, kafka-topic.sh will have to be used.

The correct command depends on the version of Kafka that is in use.

Up to 0.8.2 kafka-topics.sh --alter is the command to be used.

From 0.9.0 going forward, use kafka-configs.sh --alter

78. Explain the graceful shutdown in Kafka.

Any broker shutdown or failure will automatically be detected by the Apache cluster. In such a case, new leaders will be elected for partitions that were previously handled by that machine. This can occur due to server failure and even if it is intentionally brought down for maintenance or any configuration changes. In cases where the server is intentionally brought down, Kafka supports a graceful mechanism for stopping the server rather than just killing it.

Whenever a server is stopped:

- Kafka ensures that all of its logs are synced onto a disk to avoid needing any log recovery when it is restarted. Since log recovery takes time, this can speed up intentional restarts.
- Any partitions for which the server is the leader will be migrated to the replicas prior to shutting down. This ensures that the leadership transfer is faster, and the time during which each partition is unavailable will be reduced to a few milliseconds.

79. Can the number of partitions for a topic be reduced?

No, Kafka does not allow reducing the number of partitions for a topic. The partitions can only be increased but not decreased.

80. How can a cluster be expanded in Kafka?

In order to add a server to a Kafka cluster, it just has to be assigned a unique broker id, and Kafka has to be started on this new server. However, a new server will not automatically be assigned any of the data partitions until a new topic is created. Hence, when a new machine is added to the cluster, it becomes necessary to migrate some existing data to these machines. The partition reassignment tool can be used to move some partitions to the new broker. Kafka will add the new server as a follower of the partition that it is migrating to and allow it to completely replicate the data on that particular partition. When this data is fully replicated, the new server can join the ISR; one of the existing replicas will delete the data that it has with respect to that particular partition.

81. Explain customer serialization and deserialization in Kafka.

In Kafka, message transfer among the producer, broker, and consumers is done by making use of a standardized binary message format. The process of converting the data into a stream of bytes for the purpose of the transmission is known as serialization. Deserialization is the process of converting the bytes of arrays into the desired data format. Custom serializers are used at the producer end to let the producer know how to convert the message into byte arrays. Deserializers are used at the consumer end to convert the byte arrays back into the message.

82. What is meant by the Kafka schema registry?

For both the producers and consumers associated with a Kafka cluster, a Schema Registry is present, which stores Avro schemas. Avro schemas allow the configuration of compatibility settings between the producers and the consumers for seamless serialization and deserialization. Kafka Schema Registry is used to ensure that there is no difference in the schema that is being used by the consumer and the one that is being used by the producer. While using the Confluent schema registry in Kafka, the producers only need to send the schema ID and not the entire schema. The consumer uses the schema ID to look up the corresponding schema in the Schema Registry.

83. What can Kafka Monitoring be used to do?

- Track Resource Utilization of the system: it can be used to keep a close tab on the resources like memory, CPU, and disk utilization over time.
- Monitor threads and JVM usage: since Kafka relies on the Java garbage collector to free up memory, ensuring that the garbage collector runs frequently ensures that more activity occurs in the Kafka cluster.
- Watch statistics related to the broker, controller, and replication so that the states of partitions and replicas can be adjusted if required.
- Performance problems can be fixed quickly by finding out which applications cause excessive load and identifying performance bottlenecks.

84. How does Kafka ensure minimal data modification when data passes from the producer to the broker to the consumer?

Kafka uses a standardized binary message format that is shared by the producer, broker, and consumer to ensure that the data can pass without any modification.

85. Name the various types of Kafka producer API.

There are three types of Kafka producer API available-

- Fire and Forget
- Synchronous producer
- Asynchronous producer

86. What role does the Kafka consumer API and Kafka producer API play?

A consumer API enables an application to subscribe to one or more topics and process the stream of records provided to them.

You can see Kafka producer API play the role of a wrapper for the two producers—Sync Producer and Async Producer. The goal is to give the client access to every producer capability using a single API.

87. How can you write data from Kafka to a database?

There are two different frameworks- Connect Source and Connect Sink. You can send topics from Kafka to external databases and load the data from source databases.

88. What is the best method to determine the number of topics in a single Kafka broker?

The list command can be used to see all of the topics in a broker. Additionally, you can view subject details with the describe command.

89. Name the configuration file to be used to set up ZooKeeper properties in Kafka.

zookeeper.properties file.

90. What is the ZooKeeper ensemble?

ZooKeeper works as a coordination system for distributed systems and is a distributed system on its own. It follows a simple client-server model, where clients are the machines that make use of the service, and the servers are nodes that provide the service. The collection of ZooKeeper servers forms the ZooKeeper ensemble. Each ZooKeeper server is capable of handling a large number of clients.

91. What are Znodes?

Nodes in a ZooKeeper tree are referred to as znodes. Znodes maintain a structure that contains version numbers for data changes, acl changes, and also timestamps. The version number, along with the timestamp, allows ZooKeeper to validate the cache and ensure that updates are coordinated. The version number associated with Znode increases each time the znode's data changes.

92. What are the types of Znodes?

There are three types of Znodes, namely:

Persistence Znode: these are the znodes that remain alive even after the client who created that particular znode is disconnected. All znodes are persistent by default unless otherwise specified.

Ephemeral Znode: Ephemeral znodes remain active only until the client is alive. Ephemeral Znodes get deleted whenever the client that created them

gets disconnected from the ZooKeeper ensemble. They play an important role in the leader election.

Sequential Znode: when znodes are created, it is possible to request the ZooKeeper to add an increasing counter to the end of the path. This counter is unique to the parent znode. Sequential nodes may be persistent or ephemeral.

93. How can we create Znodes?

Znodes are created within the given path.

Syntax:

create /path/data

Flags can be used to specify whether the znode created will be persistent, ephemeral, or sequential.

create -e /path/data

creates an ephemeral znode.

create -s /path/data

creates a sequential znode.

All znodes are persistent by default.

94. How can we remove Znodes?

rmr /path

It can be used to remove the znode specified and all its children.

95. What are ZooKeeper watches?

Clients can set watches on znodes. Any changes to that particular znode trigger the watch. When a watch triggers, ZooKeeper sends the client a notification. A watch event is a one-time trigger; another watch has to be set to get further notifications. Watches are maintained locally at the ZooKeeper server to which the client is connected.

96. What is ZooKeeper quorum?

The ZooKeeper quorum is the minimum number of server nodes that must be available for client requests. Updates done to the ZooKeeper tree by the clients must be stored in the number of servers that form the quorum for a transaction to be completed successfully.

Q = 2N + 1 is the recommended number of nodes required to form an Ensemble as defined by the quorum where:

Q - number of nodes required to form a healthy ensemble.

N - number of failure nodes that can be allowed.

97. What are the benefits of a distributed application?

Reliability: the failure of a single or a few systems does not cause the whole system to fail.

Scalability: The performance of the application can be increased as per requirements by adding more machines with minor changes in the configuration of the application with no downtime.

Transparency: the complexity of the system is masked from the users, as the application shows itself as a single entity.

98. What are some disadvantages of a distributed application?

Race conditions may arise: two or more machines trying to access the same resource can cause a race condition, as the resource can only be given to a single machine at a time.

Deadlock: in the process of trying to solve race conditions, deadlocks may arise where two or more operations may end up waiting for each other to complete indefinitely.

Inconsistency may arise due to partial failure in some of the systems.

99. What is meant by the ZooKeeper Atomic Broadcast (ZAB) Protocol?

The ZAB Protocol is the core of the ZooKeeper system, which ensures that all of the servers remain in sync. The ZAB protocol guarantees reliable delivery and ensures that messages are delivered in the same order to all the machines.

100. Where else is the ZooKeeper used?

Apart from Apache Kafka, where ZooKeeper is used for maintaining the leader and followers among the nodes in a cluster for topic partitions, ZooKeeper is also used in the following places:

- Apache Storm: manages its state of being a real-time processing framework using the ZooKeeper service.

- Apache YARN uses ZooKeeper for automatic failover of the master node.
- Yahoo! Performs the coordination and failure recovery service for Yahoo! Message Broker, which manages large amounts of data for replication and delivery. It is also used by the Fetching Service for Yahoo! Crawler for the purpose of failure recovery.

101. What are ZooKeeper barriers?

In ZooKeeper, barriers are primitives that allow a group of processes to remain in synchronization at the start and the end of a computation. There is a barrier node that serves as a parent for individual process nodes.

102. Explain Cages in ZooKeeper?

Cages refer to a distributed synchronization library for ZooKeeper. If ZooKeeper is being run on a machine or on a cluster, Cages can be used to synchronize and coordinate data access, manipulation, and processing of data, configuration management, and also membership of machines within the cluster.

103. What is the daemon name for ZooKeeper?

Quorumpeermain

104. Explain the CLI in ZooKeeper.

The ZooKeeper command-line interface or the ZooKeeper CLI is used to allow interaction with the ZooKeeper ensemble. It can be used for debugging and working with different options.

To perform CLI operations, first, turn on the ZooKeeper server and then the ZooKeeper client.

The ZooKeeper CLI can be used to perform the following commands:

- Create znodes.
- Watch znodes for changes
- Create children of a znode.
- List children of a znode.
- Remove/ delete a znode.
- Fetch data and the metadata associated with a znode.

- Set data for a particular znode.
- Check the status of a znode, such as a timestamp, version number, and data length.

RabbitMQ vs. Kafka Interview Questions

Messaging systems like Apache Kafka and RabbitMQ allow you to manage big data streams in distributed computing, including tasks such as data consumption, reading, writing, processing, etc. Although both systems are equally good as they offer numerous features, they are suitable for different big data use cases. Take a look at some of the Apache Kafka interview questions based on the differences between Apache Kafka and RabbitMQ.

105. Differentiate between RabbitMQ and Apache Kafka.

RabbitMQ	Kafka
General-purpose message broker that uses variations of request/reply, point-to-point, and publish-subscribe messaging system.	High-volume streaming and publish-subscribe messaging system.
Does not support message ordering, but since RabbitMQ is a queue, messages are stored by default in a first-in-first-out format (FIFO).	Provides message ordering using partition keys.
Messages are removed from the RabbitMQ queue once consumed, and acknowledgment is provided.	Messages remain in the Kafka log for the amount of time specified by the retention period.
RabbitMQ allows specifying message priority.	No such feature is provided.

106. How does Kafka perform better than RabbitMQ?

Kafka has much higher performance than message brokers like RabbitMQ mainly due to its use of sequential disc I/O. It can generate high throughput

(millions of messages per second) with limited resources, which is crucial for big data use cases. RabbitMQ can also deal with a million messages per second, though somewhat at the cost of greater resources (around 30 nodes).

107. Does Kafka follow the same approach as RabbitMQ for message handling?

No, Kafka does not follow the same approach as RabbitMQ for message handling.

Kafka adopts the pull approach. Consumers request a specific offset for batches of messages. Kafka enables long-pooling when there are no messages past the offset, which minimizes tight loops. A pull approach is appropriate due to Kafka's partitions. Kafka uses offsets for ordering messages in a partition with no conflicting users. Users can now leverage message batching for faster message delivery and higher efficiency.

RabbitMQ implements the push approach, which imposes a prefetch restriction on users to avoid them from becoming overburdened. This can be useful for low-latency messaging. The goal of the push model is to distribute messages separately and quickly, ensuring that work is fairly distributed and messages are processed roughly in the order they arrive in the queue.

Kafka Scenario-based Interview Questions

Every interview is incomplete without scenario-based questions. Such questions help the interviewer judge your critical thinking and problem-solving skills. Check out these Apache Kafka scenario-based interview questions to leave a solid impression in your next big data job interview.

108. Suppose you are sending messages to a Kafka topic using kafkaTemplate. You come across a requirement that states that if a failure occurs while delivering messages to a Kafka topic, you must retry sending the messages on the same partition with the same offset. How can you achieve this using kafkatemplate?

If you give the key while delivering the message, it will be stored in the same partition regardless of how many times you send it. The hashed key is used by Kafka to decide which partition needs to be updated.

The only way to ensure that a failed message has the same offset when retried is to ensure that nothing is put into the topic before retrying it.

109. Assume your brokers are hosted on AWS EC2. If you're a producer or consumer outside of the Kafka cluster network, you will only be capable of reaching the brokers over their public DNS, not their private DNS. Now, assume your client (producer or consumer) is outside your Kafka cluster's network, and you can only reach the brokers via their public DNS. The private DNS of the brokers hosting the leader partitions, not the public DNS, will be returned by the broker. Unfortunately, since your client is not present on your Kafka cluster's network, they will be unable to resolve the private DNS, resulting in the LEADER NOT AVAILABLE error. How will you resolve this network error?

When you first start using Kafka brokers, you might have many listeners. Listeners are just a combination of hostname or IP, port, and protocol.

Each Kafka broker's server.properties file contains the properties listed below. The important property that will enable you to resolve this network error is advertised.listeners.

- listeners – a list of comma-separated hostnames and ports that Kafka brokers listen to.
- advertised.listeners – a list of comma-separated hostnames and ports that will be returned to clients. Only include hostnames that will be resolved at the client (producer or consumer) level, such as public DNS.
- inter.broker.listener.name – listeners used for internal traffic across brokers. These hostnames do not need to be resolved on the client side, but all of the cluster's brokers must resolve them.

- listener.security.protocol.map – lists the supported protocols for each listener.

110. Let's suppose a producer writes records to a Kafka topic at a rate of 10000 messages per second, but the consumer can only read 2500 messages per second. What are the various strategies for expanding your consumer group?

The solution to this question has two parts: topic partitions and consumer groups.

Partitions are used to split a Kafka topic. The producer's message is divided among the topic's partitions based on the message key. You can suppose that the key is chosen in such a way that messages are spread evenly between the partitions.

Consumer groups are a method of grouping consumers together to maximize a consumer application's throughput. Each consumer in a consumer group holds on to a topic partition. If the Kafka topic has four partitions and the consumer group has four consumers, each consumer will read from a single partition. If there are six partitions and four consumers, the data will be read in parallel from only four partitions. As a result, maintaining a 1-to-1 mapping of partition to the consumer in the consumer group is preferable.

Now, you can do two things to increase processing on the consumer side:

- You can increase the topic's partition count (say from existing 1 to 4).
- You can build a Kafka consumer group with four consumer instances tied to it.

This would enable the consumers to read data from the topic in parallel, allowing it to expand from 2500 to 10000 messages per second.

Kafka Interview Questions for Java Developers | Kafka Developer Interview Questions

Kafka skills are highly in-demand these days, whether you are applying for a Big Data Engineer role, a Java Developer role, or a Kafka Developer role. This section will walk you through some Apache Kafka interview questions that are crucial for all the Java Developers and Kafka Developers out there.

111. What is Kafka's producer acknowledgment? What are the various types of acknowledgment settings that Kafka provides?

A broker sends an ack or acknowledgment to the producer to verify the reception of the message. Ack level is a configuration parameter in the Producer that specifies how many acknowledgments the producer must receive from the leader before a request is considered successful. The following types of acknowledgment are available:

- acks=0

In this setting, the producer does not wait for the broker's acknowledgment. There is no way to know if the broker has received the record.

- acks=1

In this situation, the leader logs the record to its local log file and answers without waiting for all of its followers to acknowledge it. The message can only be lost in this instance if the leader fails shortly after accepting the record but before the followers have copied it; otherwise, the record would be lost.

- acks=all

A set leader in this situation waits for all in-sync replica sets to acknowledge the record. As long as one replica is alive, the record will not be lost, and the best possible guarantee will be provided. However, because a leader must wait for all followers to acknowledge before replying, the throughput is significantly lower.

112. How do you get Kafka to perform in a FIFO manner?

Kafka organizes messages into topics, which are then divided into partitions. The partition is an immutable list of ordered messages that is updated regularly. A message in the partition is uniquely recognized by a sequential number called offset. FIFO behavior is possible only within the partitions. Following the methods below will help you achieve FIFO behavior:

- To begin, we first set the enable the auto-commit property to be false:

Set enable.auto.commit=false

- We should not call the consumer.commitSync(); method after the messages have been processed.

- Then we may "subscribe" to the topic and ensure that the consumer system's register is updated.
- You should use Listener consumerRebalance, and call a consumer inside a listener.

seek(topicPartition, offset).

- The offset related to the message should be kept together with the processed message once it has been processed.

113. How is it possible for a Kafka producer to retain exactly one semantics?

Kafka transactions assist Kafka brokers and clients in achieving precisely one semantics. To accomplish this, you must specify the properties enable.idempotence=true and transactional.id=some unique id> at the producer end. In order to prepare the producer for transactions, you must also call initTransaction. If the producer (identified by producer id>) delivers the same message to Kafka more than once with these properties set, the Kafka broker identifies and de-duplicates it.

Tricky Kafka Interview Questions

Here are a few tricky Apache Kafka interview questions for the times when your interviewer decides to test your knowledge and skills a little more but in a challenging manner-

114. Can a Kafka Consumer group have more than one consumer?

Yes, a Kafka consumer group consists of one or more consumers. In general, each customer may associate with one partition. The additional Kafka consumers in the group become inactive if the number of consumers in the specific consumer group exceeds the number of partitions. So, you must always ensure that a Kafka consumer group consists of the optimum number of Kafka consumers.

115. What does it indicate if a replica stays out of ISR for a long time?

A replica staying out of ISR for a long time indicates that the follower cannot fetch data at the same rate as data accumulated by the leader.

116. Describe how you can acquire precisely one messaging from Kafka during the data production process.

To acquire exactly one messaging from Kafka during data production, you must prevent duplication both during data production and data consumption. Below are the two methods for obtaining a single semantic during data production:

- Provide one writer per partition, and whenever a network error occurs, review the most recent message in that partition to determine if your latest write was successful.
- Include a primary key (a UUID or similar identifier) in the message and de-duplicate it for the recipient.

Kafka Admin Interview Questions | Kafka Technical Interview Questions

An individual working as a Kafka Admin is responsible for building and maintaining the entire Kafka messaging infrastructure and has expertise in concepts like Kafka brokers, Kafka server, etc.

Check out these important Apache Kafka interview questions that will help you crack any Kafka Admin job interview-

117. How can you create a Kafka topic?

[root@localhost kafka_2.9.2-0.8.1.1] #bin/kafka-topics.sh –create —zookeeper localhost:2181 –replication-factor 1 –partitions 1 –topic kafkatopic

The command above generates Kafkatopic as the topic name, and also, replication:1, partition:1.

118. What is the method to create Kafka producer API?

[root@localhost kafka_2.9.2-0.8.1.1] # bin/kafka-console-producer.sh —broker-list localhost:9092 –topic kafkatopic

This will create a producer client that receives messages from the command line and publishes messages to a single Kafka broker.

119. How do you start a single Kafka Broker?

[root@localhost kafka_2.9.2-0.8.1.1] # bin/kafka-server-start.sh config/server.properties

Why is Kafka technology significant in the Big Data industry?

Apache Kafka is an open-source stream processing platform developed by the Apache Software Foundation. It is written in Scala and Java. Kafka was originally created at LinkedIn and then open-sourced in 2011. Kafka aims to provide a platform for real-time handling data feeds and can handle trillions of events on a daily basis. Here are some features of Apache Kafka that make the Kafka technology significant in the Big Data industry:

- Scalability: Apache Kafka is able to handle messages of high volume and high velocity, making it very scalable without any downtime.
- High throughput: Apache Kafka can handle thousands of messages per second. Messages coming in at high volume and high velocity do not affect the performance of Kafka.
- Low latency: Apache Kafka offers low latency, which is low as ten milliseconds.
- Fault tolerance: Kafka is able to handle failures at nodes in a cluster, ensuring that the data is kept secure and a process running is not disturbed.
- Reliability: Kafka is a distributed platform with high fault tolerance making it very reliable.
- Durability: Data on Kafka is allowed to remain more persistent on the cluster rather than on the disk, making it quite durable.
- Real-time data handling: Kafka can handle real-time data pipelines.

6. What are the implications of increasing the number of partitions in a Kafka topic?

Increasing the number of partitions in a Kafka topic can improve concurrency and throughput by allowing more consumers to read in parallel. However, it also introduces certain challenges:

- **Increased cluster overhead**: More partitions consume additional cluster resources, leading to higher network traffic for replication and increased storage requirements.
- **Potential data imbalance:** As partitions increase, data may not be evenly distributed, potentially causing some partitions to be overloaded while others remain underutilized.
- **More complex consumer group management**: With more partitions, managing consumer group assignments and tracking offsets becomes more complicated.
- **Longer rebalancing times:** When consumers join or leave, rebalancing partitions across the group can take longer, affecting overall system responsiveness.

7. Explain the four components of Kafka architecture

Kafka is a distributed architecture made up of several key components:

Broker nodes

Kafka's distributed architecture comprises several key components, one of which is the broker node. Brokers handle the heavy lifting of input/output operations and manage the durable storage of messages. They receive data from producers, store it, and make it available to consumers. Each broker is part of a Kafka cluster and has a unique ID to help with coordination.

In older versions of Kafka, the cluster is managed using Zookeeper, which ensures proper coordination between brokers and tracks metadata such as partition locations (though Kafka is moving toward an internal KRaft mode for this). Kafka brokers are highly scalable and capable of handling large volumes of read and write requests for messages across distributed systems.

ZooKeeper nodes

ZooKeeper plays a crucial role in Kafka by managing broker registration and electing the Kafka controller, which handles administrative operations for the cluster. ZooKeeper operates as a cluster itself, called an ensemble, where multiple processes work together to ensure only one broker at a time is assigned as the controller.

If the current controller fails, ZooKeeper quickly elects a new broker to take over. Although ZooKeeper has been an essential part of Kafka's architecture, Kafka is transitioning to a new KRaft mode that removes the need for ZooKeeper. ZooKeeper is an independent open-source project and not a native Kafka component.

Producers

A Kafka producer is a client application that serves as a data source for Kafka, publishing records to one or more Kafka topics via persistent TCP connections to brokers.

Multiple producers can send records to the same topic concurrently. Kafka topics are append-only, meaning that producers can write new data, but neither producers nor consumers can modify or delete existing records, which ensures data immutability.

Consumers

A Kafka consumer is a client application that subscribes to one or more Kafka topics to consume streams of records. Consumers typically work in consumer groups, where the load of reading and processing records is distributed across multiple consumers.

Each consumer tracks its progress through the stream by maintaining offsets, ensuring that no data is processed twice and unprocessed records are not lost. Kafka consumers act as the final step in the data pipeline, where records are processed or forwarded to downstream systems.

11. How does Kafka ensure message loss prevention?

Kafka employs several mechanisms to prevent message loss and ensure reliable message delivery:

1. **Manual offset commit**: Consumers can disable automatic offset commits and manually commit offsets after successfully processing messages to avoid losing unprocessed messages in case of a consumer crash.
2. **Producer acknowledgments (acks=all)**: Setting acks=all on the producer ensures that messages are acknowledged by all in-sync replicas before being considered successfully written, reducing the risk of message loss in case of broker failure.
3. **Replication (min.insync.replicas and replication.factor)**: Kafka replicates each partition's data across multiple brokers. A replication factor greater than 1 ensures fault tolerance, allowing data to remain available even if a broker fails. The min.insync.replicas setting ensures that a minimum number of replicas acknowledge a message before it is written, ensuring better data durability. For example, with a replication factor of 3, if one broker goes down, the data remains available on the other two brokers.
4. **Producer retries**: Configuring retries to a high value ensures that producers will retry sending messages if a transient failure occurs. Combined with careful configuration, like max.in.flight.requests.per.connection=1, this reduces the chance of message reordering and ensures that messages eventually get delivered without being lost.

How does Kafka help in developing microservice-based applications?

Apache Kafka is a valuable tool for building microservice architectures due to its capabilities in real-time data handling, event-driven communication, and reliable messaging. Here's how Kafka supports microservice development:

1. Event-driven architecture

- **Service decoupling**: Kafka allows services to communicate through events instead of direct calls, reducing dependencies between them. This decoupling enables services to produce and consume events independently, simplifying service evolution and maintenance.

- **Asynchronous processing**: Services can publish events and continue processing without waiting for other services to respond, improving system responsiveness and efficiency.

2. Scalability

- **Horizontal scalability**: Kafka can handle large data volumes by distributing the load across multiple brokers and partitions. Each partition can be processed by different consumer instances, allowing the system to scale horizontally.
- **Parallel consumption**: Multiple consumers can read from different partitions simultaneously, increasing throughput and performance.

3. Reliability and fault tolerance

- **Data replication**: Kafka replicates data across multiple brokers, ensuring high availability and fault tolerance. If one broker fails, another can take over with the replicated data. Kafka stores data on disk with configurable retention periods, ensuring events are not lost and can be reprocessed if necessary.

4. Data integration

- **Central data hub**: Kafka acts as a central hub for data flow within a microservice architecture, facilitating integration between various data sources and sinks. This simplifies maintaining data consistency across services.
- **Kafka Connect**: Kafka Connect provides connectors for integrating Kafka with numerous databases, key-value stores, search indexes, and other systems, streamlining data movement between microservices and external systems.

How can you reduce disk usage in Kafka?

Kafka provides several ways to effectively reduce disk usage. Here are key strategies:

1. Adjust log retention settings

Modify the log retention policy to reduce the amount of time messages are retained on disk. This can be done by adjusting `retention.ms` (time-based retention)

or retention.bytes (size-based retention) to limit how long or how much data Kafka stores before it deletes older messages.

2. Implement log compaction

- Use log compaction to retain only the most recent message for each unique key, removing outdated or redundant data. This is particularly useful for scenarios where only the latest state is relevant, reducing disk space usage without losing important information.

3. Configure cleanup policies

- Set effective cleanup policies based on your use case. You can configure both time-based (retention.ms) and size-based (retention.bytes) retention policies to delete older messages automatically and manage Kafka's disk footprint.

4. Compress messages

- Enable message compression on the producer side using formats like **GZIP**, **Snappy**, or **LZ4**. Compression reduces message size, leading to lower disk space consumption. However, it's important to consider the trade-off between reduced disk usage and increased CPU overhead due to compression and decompression.

5. Adjust log segment size

- Configure Kafka's **log segment size** (segment.bytes) to control how often log segments are rolled. Smaller segment sizes allow for more frequent log rolling, which can help in more efficient disk usage by enabling quicker deletion of older data.

6. Delete unnecessary topics or partitions

- Periodically review and delete unused or unnecessary topics and partitions. This can free up disk space and help keep Kafka's disk usage under control.

What are the differences between leader replica and follower replica in Kafka?

Leader replica

The leader replica handles all client read and write requests. It manages the partition's state and is the primary point of interaction for producers and consumers. If the leader fails, its role is transferred to one of the follower replicas to maintain availability.

Follower replica

The follower replica replicates data from the leader but does not directly handle client requests. Its role is to ensure fault tolerance by keeping an up-to-date copy of the partition's data. Starting from Kafka 2.4, under certain conditions, follower replicas can also handle read requests to help distribute the load and improve read throughput. If the leader fails, a follower is elected as the new leader to maintain data consistency and availability.

Why do we use clusters in Kafka, and what are their benefits?

A Kafka cluster is made up of multiple brokers that distribute data across several instances, allowing for scalability without downtime. These clusters are designed to minimize delays, and in case the primary cluster fails, other Kafka clusters can take over to maintain service continuity.

The architecture of a Kafka cluster includes Topics, Brokers, Producers, and Consumers. It efficiently manages data streams, making it ideal for big data applications and the development of data-driven applications.

How does Kafka ensure message ordering?

Kafka ensures message ordering in two main ways:

1. **Using message keys:** When a message is assigned a key, all messages with the same key are sent to the same partition. This ensures that messages with the same key are processed in the order they are received, as Kafka preserves ordering within each partition.
2. **Single-threaded consumer processing:** To maintain order, the consumer should process messages from a partition in a single thread. If multiple threads are used, set up separate in-memory queues for each partition to ensure that messages are processed in the correct order during concurrent handling.

What Do ISR and AR represent in Kafka? What does ISR expansion mean?

ISR (in-sync replicas)

ISR refers to the replicas that are fully synchronized with the leader replica. These replicas have the latest data and are considered reliable for both read and write operations.

AR (assigned replicas)

AR includes all replicas assigned to a partition, both in-sync and out-of-sync replicas. It represents the complete set of replicas for a partition.

ISR expansion

ISR expansion occurs when new replicas catch up with the leader and are added to the ISR list. This increases the number of up-to-date replicas, improving fault tolerance and reliability.

What is a zero-copy usage scenario in Kafka?

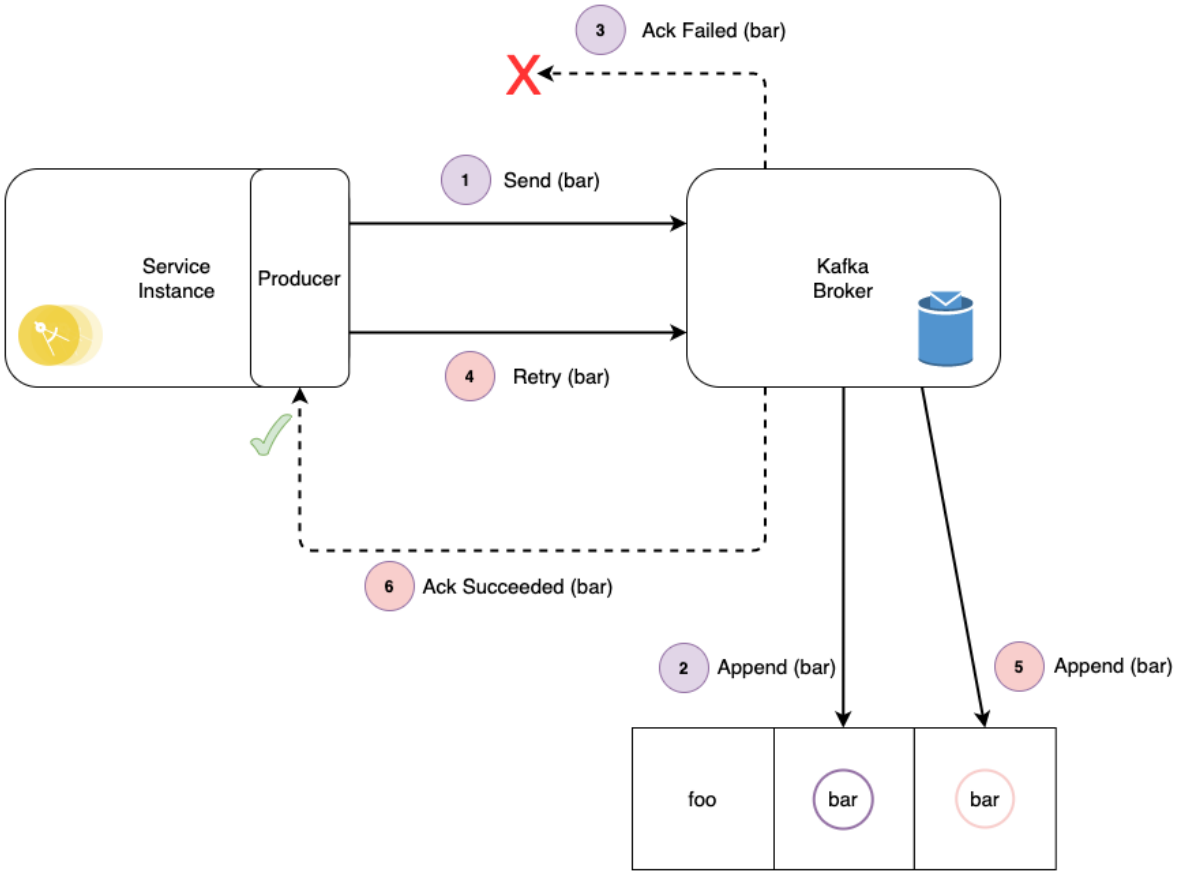
Kafka uses zero-copy to efficiently transfer large volumes of data between producers and consumers. It leverages the FileChannel.transferTo method to move data directly from the file system to network sockets without additional copying, improving performance and throughput.

This technique utilizes memory-mapped files (mmap) for reading index files, allowing data buffers to be shared between user space and kernel space, further reducing the need for extra data copying. This makes Kafka well-suited for handling large-scale real-time data streams efficiently.

Duplicate Messages

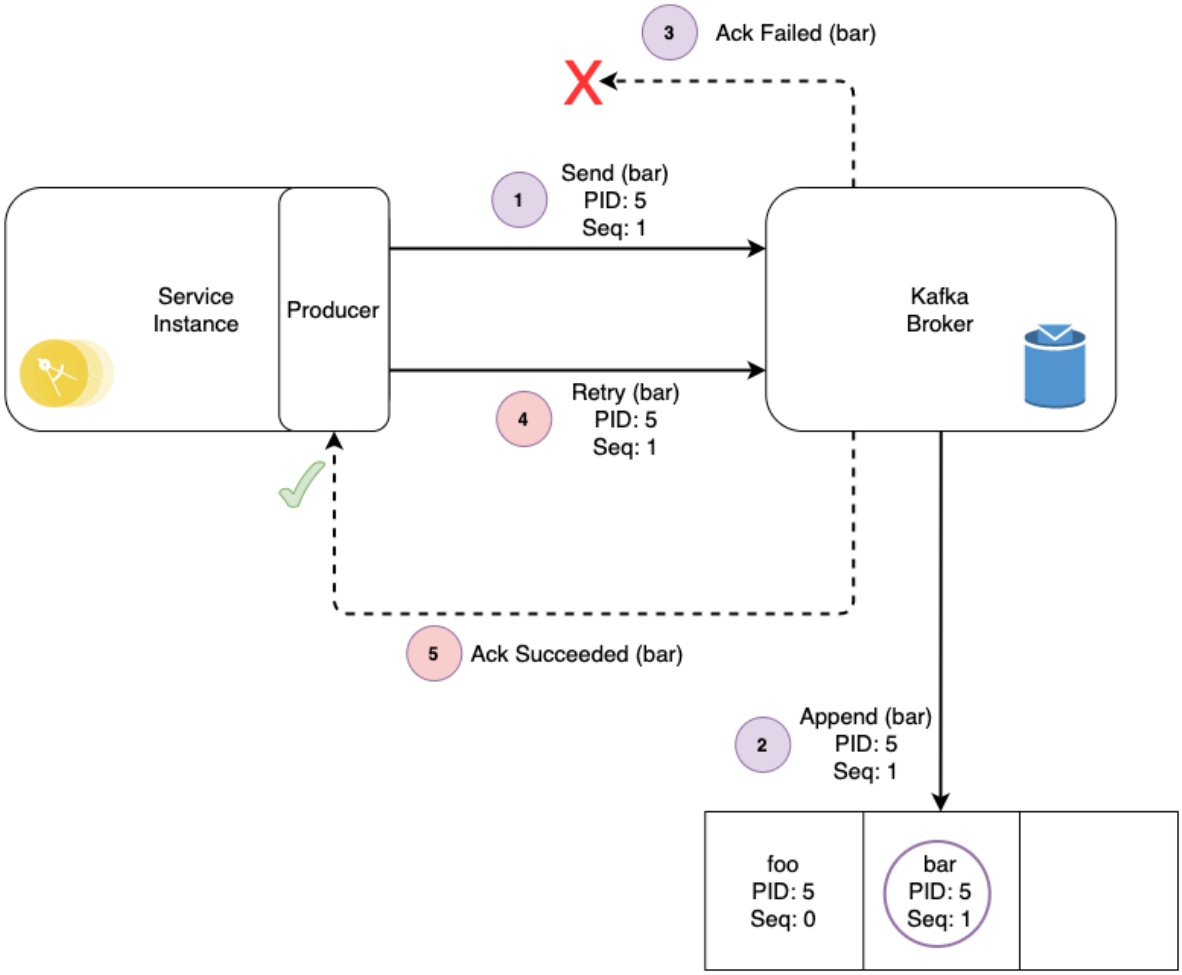
Duplicate messages can occur in the scenario where:

- A Producer attempts to write a message to a topic partition.
- The broker does not acknowledge the write due to some transient failure scenario.
- The Producer should retry as it does not know whether the write succeeded or not.
- If the Producer is not idempotent and the original write did succeed then the message would be duplicated.



Non-idempotent Producer

By configuring the Producer to be idempotent, each Producer is assigned a unique Id (PID) and each message is given a monotonically increasing sequence number. The broker tracks the PID + sequence number combination for each partition, rejecting any duplicate write requests it receives.



Idempotent Producer

Idempotent Producer Configuration

The Kafka Producer configuration **enable.idempotence** determines whether the producer may write duplicates of a retried message to the topic partition when a retryable error is thrown. Examples of such transient errors include leader not available and not enough replicas exceptions. It only applies if the retries configuration is greater than 0 (which it is by default).

To ensure the idempotent behaviour then the Producer configuration **acks** must also be set to **all**. The leader will wait until at least the minimum required number of in-sync replica partitions have acknowledged receipt of the message before itself acknowledging the message. The minimum number is based on the configuration parameter **min.insync.replicas.**

By configuring **acks** equal to **all** it favours durability and deduplicating messages over performance. The performance hit is usually considered insignificant.

If **retries** is set to zero then the Producer will not re-attempt to send the message and will instead dead-letter the message (depending on the application error handling logic). This is not recommended, as problems that are otherwise recoverable are not recovered, and the dead lettered messages instead need resolving. Meanwhile the write to the topic partition may in fact have succeeded, just not having been acknowledged by the Broker.

Unlike other concerns such as implementing an Idempotent Consumer, there is no code change required to enable the Idempotent Producer. It is just a case of providing the necessary configuration.

Producer & Consumer Timeouts

The recommendation is to leave the retries as the default (the maximum integer value) and limit retries by time, using the Producer configuration **delivery.timeout.ms** (defaulted to 2 minutes). If the message publish is being triggered by the consumption of a message, take care to ensure that this timeout does not contribute to the poll timeout being exceeded. If it does exceed the poll timeout then the Broker will remove the consumer from the consumer group as it believes it may have died, the consumer group will rebalance, and the message re-polled by the new consumer instance assigned to the partition. Meanwhile the first

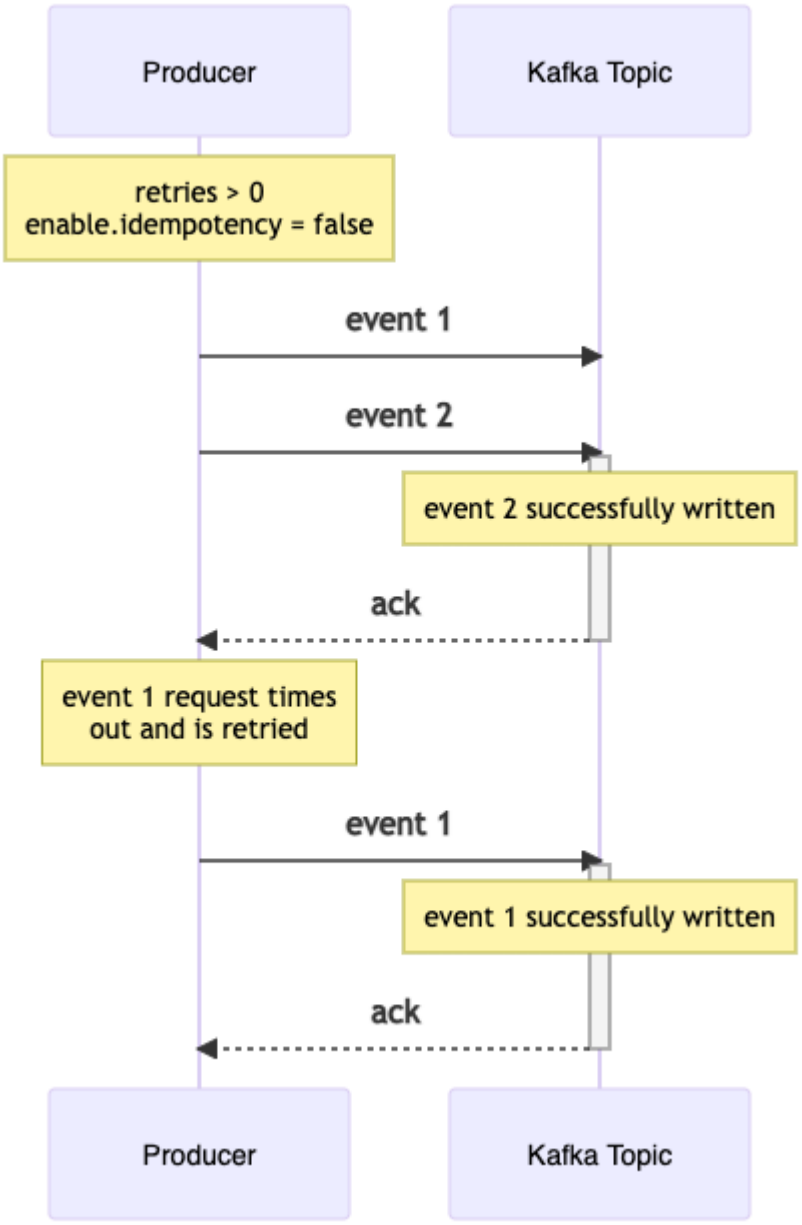
consumer instance will continue to try producing the event which may or may not succeed. In this situation downstream services will now be consuming two resulting events, themselves each unique, which would have to be mitigated for.

Guaranteed Message Ordering

The **max.in.flight.requests.per.connection** setting can be used to increase throughput by allowing the client to send multiple unacknowledged requests before blocking. However if the Producer is not Idempotent, max inflight requests is greater than 1, there is the risk of message re-ordering occurring when retrying due to transient errors. With the Idempotent Producer then up to 5 in-flight requests can be configured with message ordering guaranteed by Kafka.

For an Idempotent Producer, message ordering is guaranteed during retries by Kafka so long as the **max.in.flight.requests.per.connection** is not configured to be greater than 5. This configuration setting can be used to increase throughput by allowing the client to send multiple unacknowledged requests before blocking.

If this setting is configured to more than 5 for an Idempotent Producer, or the Producer is not Idempotent and it is configured to greater than 1, then there is the risk of message re-ordering occurring when retrying due to transient errors. This is because a later event could be written to the log before an earlier event if that earlier event is retrying, effectively switching their ordering.


Producer
Kafka Topic
retries > 0
enable.idempotency = false
event 1
event 2
event 2 successfully written
ack
event 1 request times out and is retried
event 1
event 1 successfully written
ack
Producer
Kafka Topic

Recommended Configuration

Recommended configuration for the Java Apache Kafka client library:

Configuration	Recommended Value
enable.idempotence	**true**
acks	**all**
retries	**2147483647**
delivery.timeout.ms	**120000 (2 minutes)**
max.in.flight.requests.per.connection	**< = 5**

Client Library Support

In the Java Apache client library the enable.idempotence configuration has been defaulted to false, with acks defaulting to 1, up until the most recent release of 3.0.0 (released in September 2021). In version 3.0.0 these defaults were changed to true and all respectively, favouring durability over availability. If using a pre-3.0.0 version the recommendation in the majority of cases would be to override the defaults to enable idempotency and set acks equal to all (i.e. in line with 3.0.0).

In KafkaJS (version 1.1.50 is the most recent at the time of writing) the flag to enable an idempotent Producer is marked as 'Experimental'. It is recommended to not rely on an experimental flag to give the behaviour required in Production as there is no guarantee as to its correctness or possibility of unexpected side-effects. If using KafkaJS then the design and implementation must cater for the Producer being non-idempotent.

In the librdkafka library full support for exactly-once-semantics, for which the idempotent producer is a key element, was announced as being added in April 2020 (v1.4.0). If using a binding library wrapping librdkafka then

analysis would be required to determine if this feature is now supported in that library.

How does Kafka handle backpressure when consumers lag behind?

Backpressure in kafka:

Backpressure occurs when the rate of data production exceeds the rate at which the consumers or downstream systems can process it.

Key aspects of backpressure in Kafka:

- **Consumer Lag:**

The most direct symptom of backpressure is consumer lag, which indicates the number of messages a consumer group is behind the latest message produced to a topic. Significant and increasing lag points to a consumer struggling to keep up.

- **Resource Overload:**

Unchecked backpressure can lead to resource exhaustion. For instance, if messages accumulate in a consumer's memory faster than they can be processed, it can lead to out-of-memory errors. Similarly, high message throughput can strain Kafka brokers' disk I/O, network bandwidth, and CPU.

- **Impact on Downstream Systems:**

If the consumer is acting as an intermediary to a slower downstream system (e.g., a database or another API), backpressure in Kafka can propagate to and overwhelm that system.

Strategies for managing backpressure in Kafka:

- **Consumer-side Adjustments:**
    - **Scaling Consumers:** Adding more consumer instances to a consumer group allows for parallel processing of partitions, increasing consumption throughput.
    - **Optimizing Consumer Processing:** Improving the efficiency of the consumer's message processing logic (e.g., batching operations, optimizing database queries).

    - **Flow Control Mechanisms:** Using features like pause() and resume() on the Kafka consumer API to temporarily stop fetching messages from specific partitions when a downstream system is overwhelmed.
- **Producer-side Adjustments:**
    - **Throttling Producers:** Implementing rate limiting or flow control mechanisms at the producer level to slow down the message production rate when consumer lag is detected or downstream systems are under pressure.
    - **Producer Quotas:** Configuring Kafka broker quotas to limit the data production rate for specific clients.
- **System-wide Monitoring:**
    - **Monitoring Consumer Lag:** Regularly tracking consumer lag metrics (e.g., using tools like Prometheus, Grafana, or Kafka UIs) to identify backpressure early.
    - **Broker Metrics:** Monitoring broker-level metrics like disk utilization, network throughput, and CPU usage to detect potential bottlenecks.

What is the role of the high-water mark (HW) in Kafka, and how does it impact consumers?

In Apache Kafka, the High Watermark (HW) is a crucial offset that plays a vital role in ensuring data consistency and availability. It represents the offset of the last message that has been successfully replicated to all in-sync replicas (ISRs) of a given partition.

Here's a breakdown of its significance:

- **Data Consistency:**

The HW guarantees that any message with an offset less than or equal to the HW has been successfully committed and replicated across all ISRs. This means that consumers reading up to the HW can be confident that they are consuming reliably stored and consistent data, even in the event of a broker failure.

- **Consumer Visibility:**

Consumers are only allowed to read messages up to the HW. This prevents consumers from reading messages that have not yet been fully replicated to all ISRs, which could lead to inconsistencies or data loss if the leader fails before replication is complete.

- **Replication Progress:**

The HW is maintained by the leader replica of a partition and is updated as followers acknowledge the successful replication of new messages. It effectively tracks the progress of replication across the entire In-Sync Replica set.

- **Calculation:**

The HW is calculated as the minimum Log End Offset (LEO) among all ISRs for a specific partition. The LEO represents the offset of the last message written to a replica's log.

In essence, the High Watermark acts as a boundary, ensuring that only fully replicated and committed messages are exposed to consumers, thereby maintaining the integrity and reliability of data within a Kafka cluster.

How does Kafka handle leader failure, and what happens to in-flight messages?

Kafka's handling of leader failure is an integral part of its fault-tolerance and high availability design. It is largely an automated process managed by the Kafka cluster itself, rather than requiring explicit "leader failure handlers" within client applications.

How Kafka Handles Leader Failure:

- **Leader Election:**

When a leader broker for a partition fails or becomes unreachable, the Kafka controller (a designated broker in the cluster) detects this. It then initiates a leader election process.

- **In-Sync Replicas (ISR):**

The new leader is chosen from the In-Sync Replicas (ISR) for that partition. ISRs are followers that have fully caught up with the leader's log, ensuring no data loss during a leader change.

- **Controller's Role:**

The controller updates the metadata in ZooKeeper (or the new Quorum Controllers in Kafka Raft) to reflect the new leader. This metadata is then propagated to all brokers and clients.

- **Client Reconnection:**

Kafka clients (producers and consumers) are designed to automatically detect leader changes. When a producer attempts to send messages to a failed leader, it will receive an error and automatically refresh its metadata to discover the new leader, then retry sending the messages. Similarly, consumers will fetch metadata and connect to the new leader to continue consuming.

Example Scenario (No explicit handler code required):

Consider a Kafka topic with 3 partitions and a replication factor of 3, meaning each partition has a leader and two followers.

- **Initial State:**

Broker A is the leader for Partition 0, Broker B for Partition 1, and Broker C for Partition 2.

- **Leader Failure:**

Broker A unexpectedly goes down.

- **Leader Election:**

The Kafka controller detects Broker A's failure and initiates a leader election for Partition 0. From the remaining followers (Broker B and Broker C), one is chosen as the new leader for Partition 0 (e.g., Broker B).

- **Metadata Update:**

The controller updates the cluster metadata, indicating Broker B is now the leader for Partition 0.

- **Client Adaptation:**
    - **Producers:** Producers attempting to send messages to Partition 0 will initially try to connect to Broker A. Upon failure, they will refresh their metadata and discover Broker B as the new leader, then successfully send messages to Broker B.

- **Consumers:** Consumers subscribed to Partition 0 will automatically fetch metadata updates and start consuming from Broker B.

Key takeaway: Kafka's design inherently handles leader failures through its replication and leader election mechanisms. Client applications typically do not need to implement explicit "leader failure handlers" because the Kafka client libraries are built to be resilient to such events through automatic metadata refreshing and retries. However, applications should implement robust error handling for general Kafka exceptions, including those that might occur during network issues or temporary unavailability during a leader election.

Why is unclean leader election dangerous in Kafka?

- When **unclean.leader.election.enable=true**, an out-of-sync follower can be elected as leader, potentially **losing committed messages**.
- This can cause **data inconsistencies** between producers and consumers.
- In production, **always keep this false** unless availability is more important than consistency.

How can you monitor and debug consumer lag in Kafka

Monitoring and debugging Kafka consumer lag involves understanding the difference between the latest message offset in a topic partition and the last offset consumed by a consumer group for that partition. This "lag" indicates how far behind a consumer group is in processing messages.

Monitoring Consumer Lag:

- **Kafka's Built-in Tools:**
    - The kafka-consumer-groups.sh script is the most basic way to check lag.
    - Execute: bin/kafka-consumer-groups.sh --bootstrap-server <broker> --describe --group <group-id>
    - This provides real-time lag for each partition within the specified consumer group.

- **External Monitoring Tools:**
    - **Prometheus + Grafana**: Use Kafka Exporter or JMX Exporter to expose Kafka metrics, including consumer lag, to Prometheus. Visualize these metrics using Grafana dashboards for historical data and trend analysis.
    - Burrow, Confluent Cloud Console , CMAK (Cluster Manager for Apache Kafka):

Debugging Consumer Lag:

Once lag is identified, debugging involves determining the root cause and implementing solutions**:**

- **Consumer Processing Speed:**
    - **Inefficient Consumer Logic:** Optimize the code within your consumer to process messages more quickly.
    - **Limited Resources**: Ensure consumers have sufficient CPU, memory, and network resources.
    - **External Dependencies:** If consumers rely on external services, check for bottlenecks or latency in those services**.**
- **Kafka Configuration:**
    - **Consumer Properties:** Tune consumer configurations like max.poll.records, fetch.min.bytes, and fetch.max.wait.ms to optimize message fetching.
    - **Producer Rate:** If the producer rate significantly exceeds the consumer processing capacity, lag will accumulate. Consider rate limiting producers or scaling up consumers.
- **Partitioning:**
    - **Insufficient Partitions:** If a topic has too few partitions, consumers in a group may not be able to parallelize processing effectively. Increasing the number of partitions can help distribute the load.
    - **Uneven Partition Distribution:** Ensure message distribution across partitions is relatively even to prevent hot spots and uneven lag.

- **Scaling Consumers:**
    - **Add More Consumers:** Increase the number of consumer instances within a consumer group to distribute the processing load across more threads or machines. This only works effectively if the number of consumers does not exceed the number of topic partitions**.**
- **Error Handling and Retries:**
    - **Poison Pills:** Identify and handle messages that consistently cause consumer failures or slow processing. Implement dead-letter queues or error handling mechanisms to prevent these messages from blocking consumption.
    - **Retry Mechanisms:** Implement robust retry logic for transient errors to prevent messages from being permanently stuck.
- **Question:**

What happens when a Kafka broker fails? How would you design a system to handle this?

- **Answer:**
    - Kafka's replication mechanism ensures data durability. If a broker fails, another broker in the In-Sync Replicas (ISR) becomes the leader for the affected partitions.
    - To handle failures, ensure:
        - **Replication factor:** Set to at least 2 or 3 to tolerate single or multiple broker failures. According to some technical articles, this is crucial for fault tolerance.
        - **Monitoring:** Use tools to monitor broker status and automatically restart failed brokers.
        - **Consumer rebalancing:** When a broker fails, consumer groups automatically rebalance partitions among the remaining brokers.

Question: How can you guarantee exactly-once message processing in Kafka?

- Answer:

    - Kafka 0.11 introduced transactional producers, allowing for exactly-once processing.
    - Use a transactional producer and consumer with:
        - **Idempotent producers:** Configure producers to retry sending messages without creating duplicates.
        - **Consumer transactions:** Commit consumer offsets within the same transaction as the processing logic.
    - Consider using Kafka Streams for complex stream processing with built-in exactly-once semantics
- **Question:**

Your Kafka consumer group is experiencing high lag. How would you diagnose and resolve the issue?

- **Answer:**
    - **Monitor lag:** Use metrics to track message production rate vs. consumption rate.
    - **Check partition assignment:** Verify that partitions are evenly distributed among consumers.
    - **Examine consumer configuration:** Review consumer group settings
      like fetch.min.bytes, fetch.max.wait.ms, and max.poll.records.
    - **Optimize consumer code:** Ensure efficient message processing and avoid bottlenecks in the consumer logic.
    - **Increase parallelism:** Consider adding more consumers or scaling up consumer resources.

what is schema in kafka

In Apache Kafka, a schema provides a formal description of the structure and data types of messages exchanged between producer and consumer applications. Since Kafka messages are essentially byte strings, a schema acts as a contract, defining how these bytes should be interpreted.

Key aspects of schemas in Kafka include:

- **Data Definition:**

A schema specifies the fields within a message, their respective data types (e.g., string, integer, boolean), and any nested structures.

- **Interoperability:**

It ensures that producers and consumers agree on the format of the data, facilitating seamless communication and data interpretation even across different applications or teams.

- **Data Validation:**

Schemas can be used to validate incoming messages, ensuring they conform to the expected structure and data types, which helps maintain data quality and prevent errors.

- **Schema Evolution:**

In real-world systems, data structures often evolve. Schema registries, like Confluent Schema Registry, manage schema versions and provide compatibility rules (e.g., backward, forward, or full compatibility) to handle changes while ensuring older consumers can still process new messages, or vice-versa.

- **Serialization and Deserialization:**

Schemas are crucial for serialization (converting data into bytes for transmission) and deserialization (converting bytes back into structured data) using formats like Avro, Protobuf, or JSON Schema.

what is schema registry in kafka

A Schema Registry in Kafka is a standalone service that provides a centralized repository for managing and validating schemas used by Kafka producers and consumers. It operates independently of the Kafka cluster itself.

Key functions of a Schema Registry:

- **Schema Storage and Management:**

It stores a versioned history of schemas (e.g., Avro, JSON Schema, Protobuf) used for messages in Kafka topics.

- **Schema Evolution and Compatibility:**

It supports configurable compatibility settings, allowing for controlled evolution of schemas while ensuring that new versions remain compatible with older ones, preventing runtime failures during deserialization.

- **Data Governance and Consistency:**

By enforcing schema adherence, it ensures data quality, consistency, and adherence to defined data structures across the Kafka ecosystem.

- **Serialization and Deserialization:**

Kafka client libraries integrate with the Schema Registry to handle the serialization of messages by referencing the schema and prepending a schema ID, and deserialization by retrieving the correct schema based on the ID.

How it works:

- **Producer Interaction:**

When a producer sends a message, it first interacts with the Schema Registry to check if the schema for that message type is registered. If not, it registers the schema. The producer then serializes the data using the retrieved schema and sends it to Kafka, typically including a schema ID in the message.

- **Consumer Interaction:**

When a consumer receives a message, it extracts the schema ID and uses it to retrieve the corresponding schema from the Schema Registry. This schema is then used to deserialize the message.

The Schema Registry is a critical component for building robust and maintainable data streaming applications with Kafka, especially in environments with multiple producers and consumers that need to agree on data formats.

Question: How would you handle schema changes in Kafka without causing downtime?

- **Answer:**
    - Use a schema registry (like Confluent Schema Registry) to manage schema evolution.
    - Producers can register new schemas, and consumers can handle multiple schema versions.
    - Implement schema evolution strategies (e.g., forward compatibility) to ensure compatibility between different schema versions.

- Use Kafka Connect to integrate with schema registries and manage schema changes seamlessly.

SCENARIO BASED:

Handling Duplicate Messages

Scenario: You are processing financial transactions through Kafka, and you discover that some transactions are being duplicated. How would you handle this issue?

Consumer Lag Monitoring

Scenario: Your consumer group is experiencing high lag, and you're receiving alerts. Describe the steps you would take to diagnose and resolve the issue.

Data Loss During Broker Failures

Scenario: Imagine that a broker fails and your replication factor is set to 1. What steps would you take to mitigate data loss in this situation?

Message Ordering Challenges

Scenario: You have a requirement that certain events must be processed in the order they are received. How would you ensure that ordering is maintained when using multiple partitions?

chema Evolution

Scenario: You need to update the Avro schema for a Kafka topic without downtime. How would you approach schema evolution while ensuring backward compatibility?

Tricky Aspect:

- Explain the principles of backward and forward compatibility in schema evolution, the use of default values for new fields, and testing the schema changes in a staging environment.

Consumer Group Rebalancing

- **Scenario:** During a consumer group rebalance, your consumers are experiencing delays in processing messages. What might be causing this, and how would you mitigate it?

End-to-End Latency

- **Scenario:** You notice that your end-to-end latency is higher than expected. How would you analyze and optimize this latency?

Mixing Different Data Formats
Scenario: You need to support multiple data formats (JSON, Avro) for different consumers. How would you implement a solution that efficiently manages this in Kafka?

Scaling Kafka Consumers
Scenario: Your application's load has increased significantly, and you need to scale your consumers. What considerations would you take into account before scaling?

Rolling Upgrades
Scenario: You need to perform a rolling upgrade on your Kafka cluster. What steps would you take to ensure there's no disruption in service?